

UI Shapes

v1.0.0 Manual · PRIZM GAMES

UI Shapes is a single component for Unity UI (uGUI). Add one `UIShape` to a UI element and it gives you a rounded-rectangle fill, drop shadow, inner shadow, glow, outline, bevel, textures and a background blur — all drawn analytically in a shader. No sprites, no 9-slices, no blur passes, no render textures. Everything stays crisp at any resolution or scale, and effects of the same type batch together across the whole canvas.

Contents

Requirements	Background Blur
Demo scenes	Bevel
Quick start	Progress
Shape & skew	Masking
Effects	Button states
Fill	Toggles
Texture	Reusable styles
Drop Shadow	Animation
Inner Shadow	Scripting
Glow	UPM package
Outline	Performance & platforms
	Troubleshooting

Requirements

- Unity **2022.3 LTS or newer**.
- A uGUI Canvas (`com.unity.ugui`). Nothing else.
- Graphics: Shader Model 3.0 or better — OpenGL ES 3.0+, Metal, Vulkan, DirectX 11+ and every current console. (OpenGL ES 2.0 is not supported.)
- Render pipelines: Built-in, URP and HDRP. Canvas UI looks identical on all three. Background Blur works everywhere too, though URP needs a one-time setup step described in its section.

The demo scenes label things with TextMeshPro, so import *TMP Essentials* when Unity offers to. UI Shapes itself has no TextMeshPro dependency — the runtime works fine without it.

Demo scenes

`UI Shapes/Demo/` has one subfolder per render pipeline — **Built-in RP**, **URP** and **HDRP** — and each holds a *UI Shapes Demo* scene (the full showcase) and a *Background Blur Demo* scene. Open the folder that matches your project's pipeline.

The Built-in scenes have one extra trick: they blur UI sitting behind a glass panel — a "glass over text" look that only the Built-in pipeline can do. If the demo scenes for the *other* pipelines show pink materials, that's expected: their materials use that pipeline's Lit shader and only resolve there. Just ignore (or delete) the folders you don't need.

Quick start

1. Select a UI GameObject, or make an empty one under your Canvas.
2. **Add Component → UI → UI Shape.**
3. Turn on the effects you want and tune them in the inspector. That's it — the component builds and manages its own renderers.

`UIShape` works either as the element's whole visual, or alongside a regular `Image` with **Fill** switched off — handy when you just want to wrap an existing graphic in a shadow, glow or outline.

To make it a **button** or a **toggle**, click **Add Button** or **Add Toggle** next to the Selectable field at the top of the inspector — that wires up the Selectable and lets the shape restyle itself per interaction state. See [Button state transitions](#) and [Toggles](#) for the details.

Shape & skew

Corners

Border Mode sets how the corner radius is shared out:

- **Uniform** — one radius for all four corners.
- **Round** — the radius auto-sizes to half the short side, so squares become circles and long rectangles become pills.
- **Free** — a separate radius per corner.

Radii are in canvas units. If neighbouring radii would overlap along an edge, they all scale down together — the same rule CSS `border-radius` follows.

Corner Style picks the corner shape itself:

- **Sharp** (the default for a new shape) — a plain rectangle. The radius controls are hidden, since they don't apply.

- **Rounded** — circular corner arcs.
- **Octagonal** — each corner cut off as a straight 45° chamfer.

For Rounded and Octagonal the radius values set the arc radius or chamfer size, so Uniform / Round / Free work with both. Every effect adopts the corner shape you choose — fill, outline, drop shadow, inner shadow, glow and bevel all share the same geometry, so the element reads as one coherent shape. Transitions between Sharp and the other styles animate smoothly, the corners collapsing to (or growing from) a point. All three styles cost the same on the GPU, so pick by looks, not by budget.

Skew & taper

Skew is its own module with an on/off header checkbox. Turn it on to slant the shape by a **Skew Angle** (in degrees, like italic text) along the chosen **Skew Axis** — Horizontal or Vertical. At 0° the shape is upright, and turning the module off leaves it upright no matter the angle. That means you can keep an angle set and just toggle the slant on and off, or animate the toggle per state.

Skew works with any corner style and with every effect — fill, outline, shadows, glow and bevel all slant together, still anti-aliased. It's purely visual: your layout, anchoring and RectTransform don't change.

Skew Mode decides how the two slanted edges relate to each other:

- **Parallel** — both edges share the Skew Angle, so the shape stays a parallelogram. This is the classic italic slant, and the default.
- **Mirrored** — the opposite edge takes the negative angle, giving a symmetric **trapezoid** (a keystone). Great for tapered banners, ribbons and perspective-looking panels.
- **Custom** — each edge gets its own slider. On a Horizontal skew these are **Left Angle** and **Right Angle**; on a Vertical skew, **Bottom Angle** and **Top Angle**. Match them for Parallel, mirror them for Mirrored, or set anything in between for an asymmetric trapezoid.

A trapezoid narrows as the angle grows, and the taper stops on its own once the two edges would meet — so a steep angle on a wide, short element settles into a clean triangle instead of folding through itself. Because the limit is that meeting point, it depends on the element's proportions: a tall element can take a much steeper angle than a short one.

Corner Style and Skew both animate and support per-state overrides. Angles cross-fade smoothly between button states (the slant fading to nothing when a state turns Skew off), and mode changes blend too — going from Parallel to Mirrored sweeps the second edge across rather than snapping. Soft masking and Background Blur follow the shape in every mode: a soft mask clips its children to the octagonal, skewed or tapered outline, and a blur panel takes that same shape. Clicks follow it as well, so a slanted or tapered button only responds inside its real outline (more on that under Fill).

ONE LIMITATION

A skewed soft mask assumes the mask and its canvas aren't rotated.

Effects

Each effect is a collapsible module styled like Unity's Particle System. Its **enable checkbox lives in the module header** — tick it to turn the effect on, or click the bar to expand its settings. You can expand a module while it's off to review its settings; they show read-only until you enable it.

Every effect has a colour, an opacity and a blend mode — **Normal**, **Additive** or **Multiply**. **Intensity** (0.1–10) multiplies alpha above 1 to make an effect's core denser without growing it.

Colour & gradients

Fill, Drop Shadow, Inner Shadow, Glow and Outline can each use a **Single Color** or a two-colour linear gradient: **Vertical** (top → bottom), **Horizontal** (left → right), or a **Custom Angle** gradient with its own slider (0° puts the end colour on the right, 90° on top). The gradient runs across the element's rect, and both colours carry alpha — so you can fade an effect across the shape, and a fully transparent start or end colour gives a smooth fade-in.

Colours blend perceptually, in gamma space, matching Photoshop, Figma and CSS. A white-to-black gradient reaches 50% grey in the middle, and a colour-to-transparent fade stays even instead of letting the bright colour linger. Gradients cost nothing extra — they render in the same batched draw call, work with masking, state transitions (each gradient field has its own override) and the Animator, and cross-fade smoothly between states, even between a single colour and a gradient.

Colours stay correct whether or not Unity 6's Canvas option **Vertex Color Always In Gamma Space** is on.

Fill

The shape's solid interior — a single colour or a two-colour gradient.

Whether the shape catches pointer events (needed when a Button has no Image of its own) is set at the top of the inspector, next to **Selectable**, since the two usually go together. A normal shape shows a **Raycast Target** checkbox. A shape whose Selectable lives on a *different* UI Shape — like a child checkmark — shows a **Raycast Mode** dropdown instead:

- **Follow** (default) — no hit-testing of its own; it mirrors the other shape's Highlighted / Pressed / Selected look across the whole shape, so a checkmark lights up with its toggle.
- **On** — catches pointer events on its own bounds and computes its own state.
- **Off** — catches nothing and follows nothing; purely decorative.

Hit-testing matches the shape you actually see, not its bounding rectangle. Rounded and octagonal corners, the slant and the taper are all taken into account, so a click just outside a corner or beside a slanted edge falls through to whatever's behind. Effects that reach past the shape — drop shadow and glow — aren't clickable.

Hit-testing only ever applies to the shape itself. Anything parented under it — a Button, an Image, a text label, another UI Shape — keeps its own pointer behaviour, so turning **Raycast Target** off makes the shape click-through without affecting its children, and a child sitting over a rounded corner or past the shape's edge stays clickable.

Texture

An optional texture drawn on top of the shape and clipped to its corners. It's its own module, so you can enable it independently of Fill, with its own **Blend Mode** and **Opacity** — exactly like a UI Image layered above the fill. **Texture** is the sprite to draw; leave it empty and the module draws nothing, or (once enabled without a sprite) a plain white rect at the set opacity — the same "not configured yet" state as any other module.

Texture Fit decides how the sprite fits the rect when the aspect ratios don't match:

Mode	Behaviour
Stretch (default)	Scales to fill, ignoring the texture's aspect ratio.
Aspect Fit	Keeps aspect, fits entirely inside the shape; the leftover margin is transparent (like <code>Image.preserveAspect</code>). Corner radii apply to the fitted panel, which is usually smaller than the full rect.
Aspect Fill	Keeps aspect, scales up to cover the shape, cropping the overflow.
Constant Size	A fixed on-screen texel size, from the sprite's <i>Pixels Per Unit</i> and the Canvas's <i>Reference Pixels Per Unit</i> , scaled by a Pixels Per Unit Multiplier slider that only appears in this mode (parity with <code>Image.pixelsPerUnitMultiplier</code>). The texture tiles to cover the rect, so it needs a non-atlas sprite whose texture <i>Wrap Mode</i> is <i>Repeat</i> ; the inspector warns if the assigned sprite doesn't qualify.

The texture always renders **upright** — it never shears or tapers with a skewed shape, even while the silhouette does. Under skew, Aspect Fit's panel is inscribed so it never pokes past the sheared outline, and Stretch / Aspect Fill fit against the skewed shape's on-screen bounding box. Every Texture field is a per-state style property like Fill's, so states can swap the sprite or fit mode, and the multiplier and opacity animate and cross-fade. A Texture with a sprite assigned renders in its own draw call (batched by texture, like any Image); Fill and the solid-colour effects keep sharing their draw calls regardless.

Drop Shadow

An analytic gaussian shadow behind the element. **Radius** is the blur distance, **Distance / Angle** offset it (0° = right, 90° = up), and **Spread** grows or shrinks the shadow before it's blurred. The shadow is always cut out of the shape's interior, so a transparent or semi-transparent fill (a glass panel, say) never shows its own shadow through itself.

Inner Shadow

The same parameters as Drop Shadow, but drawn inside the shape's edge. It covers the outer edge completely, so the fill colour never peeks through as a thin rim between the shadow and the background. As a result the silhouette reads about one pixel further out than the fill alone.

Glow

A centred gaussian halo, additive by default. Radius, Spread and Intensity behave as above. Like the drop shadow, it renders only outside the shape.

Outline

A stroke on the shape's edge. **Placement** sits it Inside, Outside or Centered on the edge; **Distance** offsets it further out (+); **Width** and **Softness** are in canvas units (Softness 0 = crisp). Whichever placement you pick, the stroke sits cleanly over the fill's edge and keeps an even thickness on skewed shapes.

On a square-cornered shape (Corner Radius 0), Outside and Center strokes miter to a sharp corner; a real radius grows to radius + width on the outside, the way design tools stroke a rounded rectangle.

NOTE

A *soft* Outside outline (Softness above 0) can show a very faint trace where its falloff crosses the fill's edge. Inside or Center avoid it.

Background Blur

A frosted-glass blur of everything rendered behind the shape. **Radius** is the blur strength in canvas units, **Blur Opacity** mixes between the sharp and blurred background, and **Quality** sets the sample count — High (32 taps) is smoothest, Medium (16) and Low (8) cost proportionally less and suit mobile (lower counts can show a little grain at large radii). Pair it with a semi-transparent Fill for tinted glass. The blur is clipped to the shape's corners, respects soft masks, and composes correctly where panels overlap.

Built-in render pipeline

Works out of the box. The capture happens per panel, exactly when it draws, so the blur includes the 3D world *and* any UI behind the panel — even elements on the same canvas.

Universal Render Pipeline (URP)

One-time setup: select your URP Renderer asset (listed under your active URP Asset's *Renderer List*) and add the Renderer Feature **UI Shapes Background Blur Feature**. Until you do, blur panels stay invisible and the inspector shows a reminder.

The feature is free while idle — the screen capture only runs on frames where a blur panel is actually active, so it's safe to leave installed in scenes that never blur. Its **Capture Resolution** (default Half) sets how much detail the capture keeps: a blur only uses low frequencies, so Half looks virtually identical to Full while the kernel reads a texture with 4× fewer pixels — a big mobile saving. Quarter is cheaper still and fine for large radii.

URP / HDRP CAPTURE

The background is captured once per camera, after post-processing, so an overlay-canvas panel blurs the finished world image — but **UI drawn by the camera itself, and other elements on the same canvas, are not part of the capture**. For UI-over-UI blur on URP, render the content behind the panel with a separate camera. Only the Built-in pipeline blurs same-canvas UI.

High Definition Render Pipeline (HDRP)

No setup needed. UI Shapes creates a hidden global Custom Pass volume automatically while a blur panel is active, capturing the frame after post-processing. The capture is Half resolution by default (set `UIShapesBackgroundBlurCustomPass.CaptureResolution` from code to change it). The same capture rules as URP apply.

PERFORMANCE

Background Blur is by far the most expensive effect here — a large panel can cost more GPU time than every other effect combined on mobile-class GPUs (Switch included). Each panel samples the captured background up to 32 times per pixel; the Built-in path also captures the screen once per panel, and URP/HDRP once per camera.

As a reference: a panel covering about a fifth of a 1080×2400 screen on a 2023 budget Android GPU (Adreno 613, Half capture) costs roughly **15 ms at High, 9 ms at Medium, 5 ms at Low**. Your levers, in order of impact: the URP feature's **Capture Resolution** (keep it at Half or lower on mobile — Built-in's GrabPass is always full resolution), the per-shape **Quality**, and the panel's on-screen size. It's great for menus and dialogs; budget carefully before putting it on a permanent HUD, and prefer a few large panels over many small ones. Background Blur is not supported in VR — every other effect is.

Bevel

A Photoshop-style inner bevel: a lit band along the shape's edge, with a highlight on the light-facing side and a shadow opposite, wrapping smoothly around rounded corners. **Width** is the band size in canvas units, **Depth** the shading strength, and **Softness** feathers the band's inner boundary *and* rounds the corner transitions between edges (0 = a crisp chisel with hard miter lines). **Angle** is the light direction (0° = right, 90° = up; the default 135° is Photoshop's up-left). **Invert** flips the lighting so the shape reads as pressed in rather than raised — handy for a pressed button state.

The highlight and shadow each have their own colour, opacity and blend mode. The defaults are additive white and normal black; set the shadow to *Multiply* for the classic Photoshop look (note that Multiply-blended effects use a second material and reduce batching).

Progress

Reveal just a fraction of the shape, like a UI `Image`'s *Fill Amount* — a radial timer, a linear bar, a cooldown sweep. It's a single analytic wipe with smooth anti-aliased edges and no extra draw calls, and it's **off by**

default, so you only pay for it when you opt in.

- Progress is its own collapsible module; its **header checkbox** turns it on. While off, the wipe math never runs.
- **Amount** (0–1) is how much of the shape shows. This is the value you animate or drive from script (`ProgressAmount`).
- **Method** — *Radial* is a 360° angular sweep (pie / clock); *Linear* is a straight wipe along an edge.
- **Origin** — where the fill starts. Linear: the edge it grows from (Top / Right / Bottom / Left). Radial: the angle the sweep begins at.
- **Clockwise** (radial only) reverses the direction.
- **Targets** picks which effects the wipe trims — any mix of Fill, Outline, Glow, Drop Shadow, Inner Shadow and Bevel. Effects you leave unticked always draw in full, so you can wipe the Fill while a static Outline frames the whole shape. Background Blur is never trimmed.

Progress works with masking, the Animator and button states. It combines with everything else per pixel, so a gradient, texture or bevel is revealed exactly as far as the Amount.

Masking

Mode	What it does
None	No masking.
Simple	Add a Unity <code>Mask</code> component to the same GameObject and keep Fill enabled: the rounded fill becomes the stencil that clips children. Hard (stencil) edges.
Soft	Anti-aliased rounded clipping of children with no stencil and no RenderTexture. Remove any Unity Mask on the object first.

A **Soft** mask clips child `Image` s and `RawImage` s, child UI Shapes (fills, shadows, glows and outlines all respect the mask — ideal for scroll views of UI Shape cards) and **TextMeshPro text** when a TMP package is installed (detected automatically at compile time; there's no hard TMP dependency). Soft-clipped TMP text keeps its *Mobile Distance Field* features — face, outline and underlay (drop shadow) with their material settings and keywords.

Soft masking has a few limits: only one Soft mask can be active at a time; effects that only exist in the full Distance Field shader (bevel, glow, lighting, textured face) aren't applied while text is clipped; TMP sub-mesh content (inline sprites, fallback fonts) isn't clipped; and the clip is purely visual — a clipped child that catches **raycasts** still receives pointer events, so set **Raycast Mode** to Off on elements that scroll out of view. A soft-masked text keeps its material preset: the clip is injected only at render time, so switching presets, editing material settings, saving scenes and entering play mode all behave exactly as they would without a mask.

Padding and **Softness** (Soft mode only) shape the clip area itself, with a separate value per side — **Left**, **Right**, **Top**, **Bottom**, all in canvas units.

Padding insets the clip, exactly like the padding on Unity's `RectMask2D` : a positive value pulls that side inward, so children are cut off short of the shape's edge. Negative values push the side outward instead, letting children spill past the shape — handy when the mask's visible frame is smaller than the area you want to keep visible. The corner radii are unchanged by padding (they're only clamped so they can never exceed the padded area), and on a skewed or tapered mask the inset follows the slanted edges.

Softness fades the clip out over that distance on that side instead of ending at a hard (anti-aliased) edge. A Top and Bottom softness on a scrolling list is the classic use — the rows dissolve at the top and bottom of the viewport with no gradient overlay image. A side left at 0 keeps its crisp edge, and two adjacent soft sides meet cleanly in the corner rather than doubling up and darkening it. Softness is a visual fade only: children are still clipped at the padded boundary, and culling is unaffected.

Padding and Softness are component-level settings, like Masking itself — they are the same for every button state and are not part of a saved style, though both can be driven from script (`MaskPadding` , `MaskSoftness`).

A Soft mask also **culls children that scroll fully out of view**, just like Unity's `RectMask2D` . A child Image, TMP text or UI Shape entirely outside the mask stops rendering until it scrolls back in, so a long list only pays for the rows actually on screen. This is automatic. It uses the mask's upright rectangle (grown to cover a skewed mask's slant), so a child is only culled once it's completely outside — an effect that reaches past a card's edge, like a drop shadow, keeps it alive until that too is clear. (This is a render optimization only; pointer events are unaffected, so still set **Raycast Mode** to Off on off-screen rows.)

WHICH TO USE?

Soft is the best-looking option and the right default for the common case — one scrolling list or panel of Images, UI Shapes and text. Reach for **Simple** when Soft's structure doesn't fit: **nested masks** (only one Soft mask can be active at once), a **rotated** mask, **custom-material or third-party children** (Soft only clips plain Images, RawImages, UI Shapes and TMP text — a stencil clips every descendant), or TMP text that needs the full material feature set (bevel, glow, lighting, textured face) inside the mask.

Maskable controls whether this element's *own* effects can be clipped by an ancestor Mask or RectMask2D — turn it off to let a big shadow or glow spill outside a masked parent.

Button state transitions

`UIShape` can restyle itself per Selectable state — no sprite swaps:

1. Assign your Button (or any `Selectable`) to **Selectable** — or click **Add Button / Add Toggle** next to the field to add that component here and assign it in one step. On assignment, UI Shapes sets the Selectable's own *Transition* and *Navigation* to **None** (this component drives the visuals, and Highlighted is meaningless once keyboard/controller navigation is on) and turns this shape's **Raycast Target** on so it receives clicks. Turn Navigation back on afterwards if you need controller or keyboard support.
2. A row of tabs appears — **Normal · Highlighted · Pressed · Selected · Disabled**. Normal is the component's main setup; the others are the per-state looks. (The tabs stay hidden until a Selectable is assigned.) If the Selectable is a **Toggle**, an **Off | On** bar appears above the tabs — see Toggles.

3. On a state tab, tick the checkbox next to any property you want to override there; unticked properties follow the Normal look. Turning a module's header checkbox on or off in a state is itself an override — that's how you enable or disable an effect for a state. An **orange bar** in the left gutter marks every property, module header and tab that differs from Normal, so you can see what a state changes without opening it. Toggling a module on or off on the **Normal** tab mirrors that into every state, so switching an effect on doesn't quietly leave it off everywhere else.
4. **Transition Duration** (next to the Selectable field) cross-fades between states (0 = instant). Fades use unscaled time, so they keep working while the game is paused.

Selecting a state tab previews it live on the element, and editing its values keeps the preview current — the active tab *is* the preview, so switch back to **Normal** to see the base look. Previews are never saved: they're applied at render time only and never touch the stored settings, so saving, reloading scripts or entering play mode always keeps your authored Normal values. Unticking an override snaps that property straight back to Normal, so there's nothing to reset by hand.

State precedence matches Unity's own Selectable: Disabled > Pressed > Selected > Highlighted > Normal.

Toggles

Assign a **Toggle** as the Selectable (or click **Add Toggle**) and the tab bar becomes two rows: a top **Off | On** bar for the checked side, and beneath it the usual **Normal · Highlighted · Pressed · Selected · Disabled** tabs. Every interaction state can be authored separately for the unchecked and the checked toggle.

- **Off** is exactly the ordinary state model: Off/Normal is the base look, the other Off tabs override it.
- **On/Normal** is the checked resting look — a set of overrides on top of the base, like a state. Each On state inherits from the matching Off state, with On/Normal layered in between. So any value on an On state tab resolves in this order:

1. the On state's own override, if ticked;
2. otherwise On/Normal's value, if On/Normal overrides it;
3. otherwise the matching Off state's value (which itself falls back to Off/Normal).

For example: Off/Normal fill dark grey, Off/Highlighted fill light grey. On/Highlighted starts out light grey (inherited from Off/Highlighted). Set On/Normal's fill to green and On/Highlighted follows it to green. Then override On/Highlighted itself to light green and it stops inheriting. A disabled-but-checked toggle resolves the same way (On/Disabled → On/Normal → Off/Disabled). The inspector always shows the resolved value, so you can see exactly what an un-overridden field inherits.

- Checking and unchecking cross-fade with the same **Transition Duration** as any other state change.
- Changes from code — including `SetIsOnWithoutNotify` and **Toggle Group** switching (radio groups) — are picked up automatically; there's nothing to call.

Checkmarks & following children

Add Toggle also drops in a ready-made child *Checkmark* UI Shape — grey when off, green when on — driven by the same Toggle (which starts unchecked), so a fresh toggle works right away. Restyle or delete

it freely. That's the general pattern for shape checkmarks: give a child UI Shape the *same Toggle* as its Selectable and author its On/Off looks (the default setup just overrides the Fill colour on On/Normal). A UI Shape can't go in the Toggle's own *Graphic* field — it isn't a uGUI Graphic — so use that field only if you'd rather have a plain Image checkmark that Unity fades for you.

Because the checkmark shares the parent's Selectable, it also **follows the parent's Highlighted / Pressed / Selected state across the whole parent** — hover anywhere on the toggle and the checkmark lights up too, even though it catches no pointer events itself. Any child shape whose Selectable is on another UI Shape does this automatically; it's the **Raycast Mode** dropdown's default *Follow* (set it to *On* to hit-test the child's own bounds instead, or *Off* to make it inert — see Fill).

Every tab in both rows supports overrides, live preview and style assets alike. Differences are flagged with an **orange bar** in the left gutter — next to each property, on module headers, and under the tabs — like Unity's prefab-override lines. A bar marks a value or module that differs from its baseline: an Off state (or On/Normal) compared to Normal, and an On state compared to **On/Normal** (so a checked state only flags what it changes on top of the checked resting look). Tabs and collapsed headers aggregate the bars inside them, so a glance tells you which state and which effect is customized.

Reusable styles

Any tab — Normal or a button state — can save its settings as a **style asset** and load them onto other shapes. The **Style** field at the top of each tab is the picker:

- **Save Style** (shown while no style is loaded) saves the current tab's settings as a `.asset` file and loads it right away.
- Assigning a style in the picker (drag one in, or use the object picker) applies its settings to that tab.
- While a style is loaded, editing the tab keeps the asset up to date automatically — and every other shape with the same style loaded updates live. Editing the asset directly in its own inspector does the same.
- **Unload** (or clearing the field with the Delete key) keeps the current values but stops syncing them.

A style saved from **Normal** is a complete look: load it onto another shape's Normal tab and it replaces the base settings; load it onto a state tab and it overrides *every* property (the state no longer follows Normal). A style saved from a **state** tab is a *delta* — it stores only which properties that state overrides, and their values — so loading it onto any state of any button changes only those properties and leaves everything else following that button's own Normal look. That's what makes a saved "hover style" or "pressed style" reusable across buttons with different base looks.

- The link is an authoring convenience. At runtime nothing reads the asset, so there's no performance cost, and deleting a style asset never breaks a build — shapes keep their values.
- A state with a style loaded keeps its own enables: toggling a module on the Normal tab mirrors into ordinary states but not into styled ones.
- On prefab instances, a propagated style edit shows up as ordinary property overrides — normal Unity behaviour.
- You can also create an empty style via **Assets ▶ Create ▶ UI Shapes ▶ Style**.

Animation

All settings are plain serialized fields, so the **Animator** can drive them directly — corner radius, colours, shadow distance, and the rest. Animated values layer *under* state overrides: animation drives the base look, and states override on top.

Scripting

Everything lives in the `PrizmGames.UIShapes` namespace:

```
using PrizmGames.UIShapes;

var shape = GetComponent<UIShape>();
shape.CornerRadius = 24f;           // properties push to the shader automatically
shape.ShadowOpacity = 0.6f;
shape.SetBorderMode(UIShape.BorderMode.Round);
shape.Glow = true;
```

- PascalCase properties (`CornerRadius` , `FillColor` , `ShadowRadius` , ...) mark the component dirty on write. Gradients: `FillGradient = UIShape.GradientMode.Vertical` , `FillColor2` , `FillGradientAngle` (the same trio for `Shadow...` , `InnerShadow...` , `Glow...` and `Outline...`). Texture: `shape.Texture = true` , `TextureSprite` for the sprite, `shape.SetTextureFit(UIShape.TextureFit.AspectFit)` for the fit, `TexturePixelsPerUnitMultiplier` to scale Constant Size's tile, and `TextureBlendMode` / `TextureOpacity` for how it composites over Fill.
- Progress: enable it with `Progress = true` , then drive `ProgressAmount` (0–1) each frame — e.g. `shape.ProgressAmount = timeLeft / duration;` for a radial timer. `ProgressClockwise` plus the `SetProgressMethod` / `SetProgressOrigin` / `SetProgressTargets` setters configure the wipe.
- Shape geometry: `shape.SetCornerStyle(UIShape.CornerStyle.Octagonal)` ; enable the slant with `shape.Skew = true` , then `shape.SetSkewAxis(UIShape.SkewAxis.Vertical)` and `shape.SkewAngle = 15f` . For a trapezoid, `shape.SetSkewMode(UIShape.SkewMode.Mirrored)` , or `SetSkewMode(UIShape.SkewMode.Custom)` plus `shape.SkewAngle2 = -5f` to angle each edge separately.
- Enum-valued settings use a `Set...` method (`SetBorderMode` , `SetCornerStyle` , `SetSkewAxis` , `SetSkewMode` , `SetTextureFit` , `SetProgressMethod` , ...) rather than a property, because a property would collide with the enum type's own name.
- If you write the public camelCase *fields* directly (from your own editor tooling, say), call `shape.SetDirty()` afterwards.
- `shape.CurrentState` reads the active transition state; `shape.GetStyle(state)` returns an Off-column override block and `shape.GetOnStyle(state)` the checked column's (`GetOnStyle(Normal)` is the On/Normal block, also reachable as `shape.on`).

- `UIShapeStyle` supports full-snapshot workflows: `Capture` , `CopyFrom` , `ApplyTo` , `Merge` , `SetValues` , `ApplyBlend` .

Using UI Shapes as a UPM package

The `UI Shapes` folder is also a UPM package (`com.prizmgames.uishapes`). To reference it from another project, add this to that project's `Packages/manifest.json` :

```
"com.prizmgames.uishapes": "file:../path/to/UI Shapes/Assets/UI Shapes"
```

or, from a git repository:

```
"com.prizmgames.uishapes": "https://github.com/<you>/<repo>.git?path=/Assets/UI Shapes"
```

Performance & platform notes

How it renders

Each enabled effect draws one quad, and all its parameters travel in vertex channels. Effects of the same kind share one static material, so same-kind effects across every shape on a canvas batch together — a canvas costs at most one draw call per distinct effect kind in use. (Multiply-blended and skewed effects each use their own material.) Each material's shader contains only that effect's math, which is what keeps the per-pixel cost low on mobile GPUs. The component adds `TexCoord1–3` to the canvas's additional shader channels automatically.

General advice

- Keep effect-heavy UI on its own Canvas. The extra vertex channels apply canvas-wide, and isolating dynamic UI is good uGUI practice anyway.
- Shadow and glow **Radius** directly set how many pixels the effect covers (fill rate). On mobile, VR and Switch, prefer proportionate radii over full-screen glows, especially on large scrolling lists.
- **Overlap adds nothing special — but covered pixels still pay.** Transparent UI has no occlusion, so every effect quad costs its full on-screen area whether it ends up visible or buried: total GPU cost is simply the sum of every quad's coverage. Draw calls stay low even for many overlapping shapes (same-kind effects batch), and a screenful of static shapes costs near-zero CPU — the budget to watch is painted area. Put shadows on containers rather than on every element inside them, and let soft-masked scroll views do their thing: rows scrolled out of view are culled entirely.

Platforms

Mobile (iOS / Android). Fully supported on GLES 3.0+, Metal and Vulkan. The analytic approach avoids the classic mobile killers — render textures, blur passes, per-element materials. As a scale reference, on a 2023 budget Android phone (Adreno 613, 1080×2400) a large panel covering ~20% of the screen measures roughly 1 ms of GPU time for Fill or Outline and ~3 ms for each gaussian effect (Drop Shadow, Inner Shadow, Glow, Bevel). Cost scales with on-screen area, so typical buttons and cards are far cheaper. Background Blur has its own budget — see its section.

VR. Both shaders include the stereo-instancing macros and work with Single Pass Instanced rendering. Note that Unity doesn't render *Screen Space – Overlay* canvases in VR — use World Space (or Screen Space – Camera) canvases, as with any uGUI content.

Consoles (PlayStation, Xbox, Switch). No platform-specific code; the shaders compile through Unity's standard pipeline for every target. On Switch, treat performance like a mid-range mobile GPU — the fill-rate advice above applies.

Builds. The shaders are referenced by the component itself, so they're always included in player builds — no *Always Included Shaders* or Resources setup needed. The soft-mask scan and every per-frame path are allocation-free.

Troubleshooting

Symptom	Fix
"Shader not found" error in the console	The shader files under <code>UI Shapes/Shaders/</code> were moved or deleted — reimport the package.
Background Blur does nothing on URP	Add the UI Shapes Background Blur Feature to your URP Renderer asset (see Background Blur). The inspector shows the same reminder on the affected element.
Background Blur doesn't blur other UI on URP/HDRP	By design: the capture is taken after the camera finishes, so UI drawn by that camera (or on the same overlay canvas) isn't in it. Only Built-in blurs same-canvas UI.
Background Blur shows a sharp background	The blur shader was imported before its include file and Unity cached the stale compile. Right-click <code>UI Shapes/Shaders</code> and choose <i>Reimport</i> .
Effects invisible in VR	The canvas is Screen Space – Overlay; switch it to World Space.
Soft-clipped TMP text is missing bevel / glow / lighting	By design: while clipped, text uses a Mobile Distance Field shader (face, outline and underlay are kept). If you need the full Distance Field extras inside a mask, use Simple (stencil) masking.
TMP text isn't clipped at all	The TMP integration compiles only when a TMP package is present (<code>com.unity.textmeshpro</code> , or Unity 6's built-in TMP in uGUI 2.x). Check that TMP is installed.
A child Image or RawImage isn't soft-clipped	Images and RawImages with a custom material are deliberately skipped (the clip needs to swap the material). Remove the custom material, or use Simple masking.
Button states don't apply in play mode	Check that a Selectable is assigned and its own <i>Transition</i> is None (the inspector hints at both).
Hidden objects named <code>[UIShape] ...</code>	These are the internal effect renderers. They're never saved into your scene or prefab and rebuild automatically — don't edit them.